

Circuit Breaker: Resilience4j

1. What Problem Does Circuit Breaker Solve?

Suppose Service A calls Service B:

```
Service A
  ↓
Service B
```

Now B becomes unhealthy:

```
Service A
  ↓
Service B ✕
```

If A continues making requests:

```
Request 1 → timeout
Request 2 → timeout
Request 3 → timeout
...
Request 1000 → timeout
```

A starts consuming:

- threads
- connections
- memory
- CPU
- request capacity

Eventually **Service A itself may become unhealthy**.

This is called a **cascading failure**.

A circuit breaker prevents A from continuously calling an unhealthy B.

2. Basic Idea

A circuit breaker sits between the caller and downstream service:

```
Service A
  ↓
Circuit Breaker
  ↓
Service B
```

It monitors failures.

If failures cross a configured threshold:

```
CLOSED
  ↓
too many failures
  ↓
OPEN
```

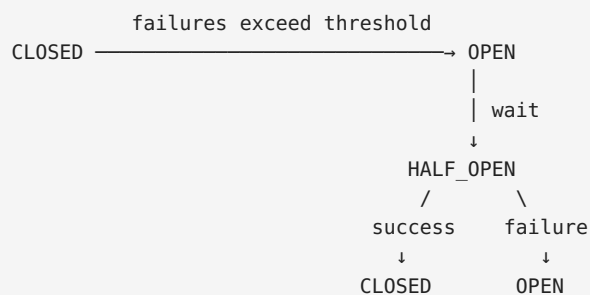
When OPEN:

```
Service A
  ↓
Circuit Breaker
  X
Service B
```

The call is rejected **without actually calling B**.

3. Three Circuit Breaker States

This is the most important thing to memorize.



CLOSED

Normal operation.

```
A → Circuit Breaker → B
```

Requests are allowed through.

The breaker keeps recording outcomes.

Example:

```
Success
Success
Success
Failure
Success
```

OPEN

Too many failures occurred.

```
A → Circuit Breaker → ✕ B
```

The circuit is open, so calls fail fast instead of calling B.

This protects both:

- the caller
- the failing downstream service

HALF_OPEN

After a configured wait period, the breaker allows a limited number of test calls.

```
OPEN
↓
wait duration
↓
HALF_OPEN
↓
test request
```

If B is healthy:

```
HALF_OPEN → CLOSED
```

If B is still failing:

```
HALF_OPEN → OPEN
```

4. Why Is It Called a Circuit Breaker?

Think of an electrical circuit.

Normal:

```
CLOSED → electricity flows
```

Failure:

```
OPEN → electricity stops
```

Same idea:

```
CLOSED → requests flow
OPEN    → requests stop
```

5. Resilience4j

Resilience4j is a lightweight fault-tolerance library for Java.

It provides several resilience patterns:

```
Resilience4j
├─ Circuit Breaker
├─ Retry
├─ Rate Limiter
├─ Bulkhead
├─ Time Limiter
└─ Cache
```

Don't confuse:

Circuit Breaker ≠ Retry

They're different patterns and are often combined carefully.

6. Spring Boot + Resilience4j

Suppose:

```
Order Service → Payment Service
```

A simple Spring implementation can look like:

```
@Service
public class PaymentService {

    @CircuitBreaker(
        name = "paymentService",
        fallbackMethod = "paymentFallback"
    )
    public PaymentResponse processPayment(
        PaymentRequest request) {

        return paymentClient.processPayment(request);
    }

    public PaymentResponse paymentFallback(
        PaymentRequest request,
        Throwable throwable) {

        return new PaymentResponse(
            null,
            "PAYMENT_SERVICE_UNAVAILABLE"
        );
    }
}
```

Conceptually:

```
processPayment()
    ↓
Circuit Breaker
    ↓
Payment Service
```

If the circuit opens:

```
processPayment()  
  ↓  
Circuit Breaker OPEN  
  ↓  
fallback
```

7. Fallback

A fallback provides an alternative behavior when the downstream operation fails or the circuit is open.

Example:

```
public PaymentResponse paymentFallback(  
    PaymentRequest request,  
    Throwable throwable) {  
  
    return new PaymentResponse(  
        null,  
        "TEMPORARILY_UNAVAILABLE"  
    );  
}
```

But be careful:

Fallback should not pretend that an operation succeeded when it did not.

For a payment:

```
Don't return:  
PAYMENT_SUCCESS
```

just because the provider is unavailable.

A safer state might be:

```
UNKNOWN  
PROCESSING  
TEMPORARILY_UNAVAILABLE
```

depending on your business model.

8. Circuit Breaker Configuration

Typical properties include concepts such as:

```
resilience4j:  
  circuitbreaker:  
    instances:  
      paymentService:  
        slidingWindowSize: 10  
        failureRateThreshold: 50  
        waitDurationInOpenState: 10s  
        permittedNumberOfCallsInHalfOpenState: 3
```

Meaning conceptually:

slidingWindowSize

How many recent calls are considered when evaluating failures.

failureRateThreshold

Failure percentage at which the breaker opens.

Example:

```
10 calls
5 failures
= 50%
```

If threshold is 50%, the circuit may open once the minimum-call criteria are met.

waitDurationInOpenState

How long to remain OPEN before attempting HALF_OPEN.

permittedNumberOfCallsInHalfOpenState

How many test calls are allowed during HALF_OPEN.

9. Sliding Window

Resilience4j supports sliding-window concepts for measuring failures.

Imagine:

```
Recent calls:
S S F F F S F F S F
```

The circuit breaker calculates failure rate over the configured window.

This is better than permanently counting all historical failures.

You generally care about **recent behavior**.

10. Minimum Number of Calls

One subtle but important point:

You generally don't want a circuit to open because of:

```
1 request
1 failure
```

There is usually a minimum number of calls before the failure rate is evaluated meaningfully.

This prevents the system from opening the circuit based on insufficient data.

11. Circuit Breaker vs Retry

Very common interview question.

Retry

“The request failed. Try it again.”

```
Call
↓
failure
↓
retry
↓
call again
```

Circuit Breaker

“This service is repeatedly failing. Stop calling it for a while.”

```
Repeated failures
↓
Circuit OPEN
↓
fail fast
```

They solve different problems.

12. Combining Retry + Circuit Breaker

You can combine them, but ordering matters.

Conceptually:

```
Request
↓
Retry
↓
Circuit Breaker
↓
Downstream
```

or another ordering depending on the desired semantics/configuration.

The important point is:

Don't blindly add retries and circuit breakers without understanding how they interact.

If 100 requests each retry 3 times, you could create:

```
100 original requests
+
300 retries
=
```

400 downstream calls

That can make an outage worse.

13. Payment Scenario

Imagine:

```
Application
  ↓
Payment Provider
```

Provider becomes unavailable.

Without Circuit Breaker:

```
1000 requests
  ↓
1000 timeouts
  ↓
threads waiting
  ↓
connection pool pressure
  ↓
application degradation
```

With Circuit Breaker:

```
Provider failures
  ↓
Circuit opens
  ↓
Requests fail fast
  ↓
No new provider calls
```

This is particularly valuable for latency-sensitive systems.

14. But Circuit Breaker Does NOT Fix the Payment

This is an important distinction.

Suppose:

```
Payment request
  ↓
Provider
  ↓
timeout
```

Circuit breaker can stop future calls when the failure threshold is reached.

But it does **not tell you whether the timed-out payment succeeded**.

You still need application-level handling:

```
Circuit Breaker
+
Transaction state
+
Idempotency
+
Reconciliation/status inquiry
```

This is where fault tolerance and business correctness differ.

15. Circuit Breaker vs Timeout

These are also different.

Timeout

Controls:

How long should I wait for one call?

Circuit Breaker

Controls:

Should I continue making calls to a repeatedly failing service?

Example:

```
One slow request
→ timeout

Repeated downstream failures
→ circuit breaker
```

Usually you want both.

```
Caller
↓
Circuit Breaker
↓
HTTP client timeout
↓
Service
```

16. Circuit Breaker vs Bulkhead

Another common interview question.

Circuit Breaker

Stops calls when failure rate becomes unacceptable.

Bulkhead

Limits how much resource one dependency/workload can consume.

Imagine:

```
Application
├─ Payment calls
├─ Customer calls
└─ Notification calls
```

If Payment calls consume all available threads, notifications may suffer.

A bulkhead can isolate resource usage.

Conceptually:

```
Payment → pool A
Customer → pool B
Notification → pool C
```

This prevents one dependency from exhausting the entire application.

17. Circuit Breaker vs Rate Limiter

Rate Limiter

Controls:

┆ How many requests are allowed?

Example:

```
100 requests/sec
```

Circuit Breaker

Controls:

┆ Are we currently allowed to call this failing dependency?

Different purpose.

18. Circuit Breaker vs Load Balancer

Don't confuse these.

Load Balancer

Chooses:

```
Instance A
Instance B
Instance C
```

Circuit Breaker

Determines whether calls to a failing dependency should be stopped.

They can work together:

```
Caller
↓
Circuit Breaker
↓
Load Balancer
↓
Service instances
```

Exact ordering depends on the architecture.

19. Common States Interviewers Expect

You should be able to explain:

```
CLOSED
↓
Failures exceed threshold
↓
OPEN
↓
Wait
↓
HALF_OPEN
├─ healthy → CLOSED
└─ failing → OPEN
```

Don't forget **HALF_OPEN**.

Many candidates remember only:

```
open / closed
```

20. Important Configuration Concepts

You don't need to memorize every Resilience4j property, but understand:

Concept	Meaning
Failure rate threshold	When failure percentage causes OPEN
Sliding window	Recent calls used for measurement
Minimum calls	Enough calls before evaluating failure rate
Wait duration	Time spent OPEN
Half-open calls	Test calls after OPEN
Slow-call threshold	Detect excessively slow calls
Slow-call rate	Percentage of slow calls that can trigger opening

Resilience4j can consider **slow calls** in addition to explicit failures.

That's a good 3-year-level detail.

21. Slow Call Scenario

Suppose the provider isn't returning errors but is extremely slow:

```
Call 1 → 15 sec  
Call 2 → 18 sec  
Call 3 → 20 sec  
...
```

Even though they're technically successful, they can still exhaust resources.

Circuit Breaker can be configured to consider **slow-call rate**.

So circuit breakers aren't only about HTTP 500 errors.

22. Monitoring

In production, you should monitor:

```
Circuit state  
Failure rate  
Slow-call rate  
Number of calls  
Rejected calls  
Fallback calls
```

Spring Boot applications commonly expose resilience metrics through their monitoring stack when configured appropriately.

This lets operations see:

```
Payment provider  
↓  
Failure rate ↑  
↓  
Circuit OPEN
```

rather than discovering the issue only from user complaints.

23. What Happens When Circuit Is OPEN?

Suppose:

```
Circuit = OPEN
```

New request comes in:

```
Client
↓
Circuit Breaker
↓
OPEN
```

The request should normally **fail fast** or invoke the configured fallback rather than call the downstream service.

This is one of the main benefits:

■ We don't waste resources waiting for a dependency we already know is unhealthy.

24. Example With a Feign Client

A common Spring setup can combine Feign and Resilience4j depending on the application's Spring Cloud configuration.

Conceptually:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @PostMapping("/payments")
    PaymentResponse process(
        @RequestBody PaymentRequest request);
}
```

Then the service layer handles resilience:

```
@CircuitBreaker(
    name = "paymentService",
    fallbackMethod = "fallback"
)
public PaymentResponse processPayment(
    PaymentRequest request) {

    return paymentClient.process(request);
}
```

Architecture:

```
Service
↓
Circuit Breaker
↓
Feign
↓
HTTP
↓
Payment Service
```

25. Should the Circuit Breaker Go Around Everything?

No.

You normally apply it around the **remote operation/dependency** where failure isolation is needed.

You generally don't want to wrap every tiny internal Java method with a circuit breaker.

Good candidate:

```
Service
↓
External API
```

Not:

```
Service
↓
String formatting method
```

26. Interview Scenario

Interviewer:

“Payment service is returning 503 for the last 5 minutes. How does Resilience4j help?”

A good answer:

“The circuit breaker monitors calls to the payment service. If the configured failure criteria are exceeded, it transitions from CLOSED to OPEN. While OPEN, calls fail fast instead of continuing to hit the unhealthy service. After the configured wait duration, it moves to HALF_OPEN and permits a limited number of test calls. If those succeed, it closes the circuit; if they fail, it returns to OPEN. This prevents repeated failures from consuming application resources and reduces cascading failure.”

27. Another Interview Scenario

“Does Circuit Breaker retry the request?”

No.

Circuit breaker and retry are separate mechanisms.

```
Retry
→ attempt again

Circuit Breaker
→ stop attempting when dependency is unhealthy
```

They can be combined, but each has a different responsibility.

28. Another Important Question

“Does Circuit Breaker work across multiple application instances?”

The answer depends on the implementation/configuration.

A typical Resilience4j circuit breaker is **in-process**.

If you have:

```
Instance 1 → circuit OPEN  
Instance 2 → circuit CLOSED  
Instance 3 → circuit CLOSED
```

the state isn't automatically globally shared among instances.

This is an important distributed-system distinction.

Don't think:

“One circuit breaker controls the entire cluster.”

Usually each application instance has its own breaker state unless you deliberately introduce a shared/distributed mechanism.

29. BPS Questions — Investigate Your Codebase

Now connect what you've learned to BPS yourself:

1. Does BPS use Resilience4j anywhere?
2. Which downstream calls have circuit breakers?
3. Is the circuit breaker around Hydra/Chase calls?
4. What counts as a failure?
5. Are timeouts counted as failures?
6. Is slow-call detection configured?
7. What are the configured failure thresholds?
8. What are the CLOSED/OPEN/HALF_OPEN settings?
9. Is there a fallback?
10. What does the fallback return for a payment failure?
11. Is retry also configured?
12. How do retry and circuit breaker interact?
13. What happens to a payment when the circuit is OPEN?
14. How is the circuit state monitored?
15. Is the circuit breaker instance-local or shared across deployments?

Interview-ready answer

“A circuit breaker is a resilience pattern used to prevent repeated calls to an unhealthy downstream service. It normally operates in three states: **CLOSED**, where calls flow normally; **OPEN**, where calls fail fast without reaching the downstream service; and **HALF_OPEN**, where a limited number of test calls determine whether the dependency has recovered. Resilience4j provides circuit-

breaker functionality for Java applications and can also provide retry, rate limiting, bulkhead and time-limiter capabilities. Circuit breakers are different from retries: retry attempts the operation again, while a circuit breaker stops calls when a dependency is repeatedly failing. In a payment system, a circuit breaker protects the application from cascading failures, but it does not determine the final business outcome of a timed-out payment, so idempotency, transaction state and reconciliation may still be required.”

What to memorize

```
Circuit Breaker
  ↓
CLOSED
  ↓ failures
OPEN
  ↓ wait
HALF_OPEN
  ↓      ↓
success  failure
  ↓      ↓
CLOSED   OPEN
```

And the key distinction:

Timeout = stop waiting for one call **Retry = try again** **Circuit Breaker = stop calling a failing dependency** **Bulkhead = limit resource consumption** **Rate Limiter = limit request rate**